

PumpScript

Design Specification Document — Part 1

CSE 341 · Concepts of Programming Languages

Gebze Technical University · 8 May 2026

Student Name Yusuf Eren Nalbant
Student No 220104004049
Language PumpScript — A DSL for Workout Routines
Deliverable D1 — Design Specification (Part 1)

Contents

1	4.1	Language Overview	2
	1.1	What is PumpScript?	2
	1.2	Sample Program	2
	1.3	Language Evaluation Criteria (Sebesta Ch. 1)	3
2	4.2	Lexical Structure	4
	2.1	Token Categories	4
	2.2	Design Notes	4
	2.3	Annotated Token Stream Example	4
3	4.3	Syntax	6
	3.1	EBNF Notation Guide	6
	3.2	Complete Grammar	6
	3.3	Ambiguity Resolution	7
	3.4	Parse Tree Examples	7
4	4.6	Names, Binding, Scope, and Lifetime	9
	4.1	4.6.1 Legal Identifiers	9
	4.2	4.6.2 Compile-time vs. Run-time Bindings	9
	4.3	4.6.3 Scoping Rule: Static (Lexical) Scoping	9
	4.4	4.6.4 Variable Lifetime	10
	4.5	Summary	11

4.1 Language Overview

What is PumpScript?

PumpScript is a domain-specific language (DSL) designed for defining, organizing, and managing workout routines. Its intended users are athletes, fitness coaches, and enthusiasts who want to describe training sessions in a structured, readable format *without requiring any programming expertise*.

The name *PumpScript* is drawn from the fitness term *pump* — the sensation of muscles filling with blood during exercise — chosen to reflect the language’s tight focus on the workout domain.

With PumpScript a user can:

- Define individual exercises as typed records (**exercise**), each carrying its own sets, reps, weight, muscle group, and completion status.
- Group exercises into parametrized routines (**routine**), callable with different arguments for different athletes.
- Specify rest periods between exercises (**rest N seconds**), a first-class domain concept absent from general-purpose languages.
- Apply conditional logic (**if/else**) and repetition (**repeat N times**) to build dynamic training plans.

Sample Program

```
1 // Chest day: each exercise carries its own weight
2 routine chest_day(sets: int) {
3     exercise bench_press {
4         sets:      sets
5         reps:      10
6         weight:    80.0
7         group:     "chest"
8         completed: false
9     }
10    rest 60 seconds
11    exercise chest_fly {
12        sets:      sets
13        reps:      12
14        weight:    20.0
15        group:     "chest"
16        completed: false
17    }
18 }
19
20 if completed == false {
21     repeat 3 times {
22         chest_day(4)
23     }
24 }
```

Listing 1: A complete PumpScript workout program

Why weight is per-exercise, not a routine parameter

In real training programs each movement uses a different load. A single **weight** parameter shared across all exercises would be unrealistic and error-prone. Placing **weight** inside each **exercise** block supports **reliability** (Sebesta §1.3.3): the language structure mirrors domain reality.

Language Evaluation Criteria (Sebesta Ch. 1)

Criterion	Priority	Justification
Readability (§1.3.1)	High	Keywords (exercise , rest , repeat , routine) mirror fitness vocabulary. A non-programmer reads rest 90 seconds and understands instantly.
Writability (§1.3.2)	High	No constructors, memory management, or import statements. Declaring an exercise requires only its relevant fields.
Reliability (§1.3.3)	Medium	Static typing catches errors before execution. Per-exercise weight reduces accidental misuse.
Cost (§1.3.4)	Sacrificed	PumpScript is not general-purpose. It cannot do file I/O, networking, or anything outside the workout domain — by design.

Key Trade-off: Generality vs. Simplicity

The central trade-off is **generality vs. simplicity**. Python could express every PumpScript program — but would require the user to learn dictionaries, functions, loops, and type annotations. PumpScript collapses all that complexity into domain concepts the user already knows.

4.2 Lexical Structure

The PumpScript lexer reads a source file character by character and produces a flat stream of tokens. Whitespace and comments are discarded. The lexer applies the **maximal munch** rule: always produce the longest possible token at each position.

Token Categories

Category	Pattern / Regex	Examples
KEYWORD	<code>exercise routine rest repeat times if else seconds true false int float string bool</code>	<code>exercise, routine, rest, repeat</code>
IDENT	<code>[a-zA-Z_][a-zA-Z0-9_]*</code> (must not match a keyword)	<code>squat, leg_day, bench_press</code>
INT_LIT	<code>[0-9]+</code>	<code>4, 8, 90</code>
FLOAT_LIT	<code>[0-9]+\.[0-9]+</code>	<code>100.0, 80.0</code>
STRING_LIT	<code>"[^"]*"</code>	<code>"legs", "chest"</code>
BOOL_LIT	Lexed as KEYWORD; promoted by parser	<code>true, false</code>
OP	<code>== != >= <= > < + - * /</code>	<code>==, !=, +</code>
SEP	<code>{ } () : ,</code>	<code>{, }, (, :</code>
COMMENT	<code>//[^\\n]*</code> (discarded)	<code>// leg day</code>

Design Notes

Keywords vs. Identifiers. Every identifier-shaped token is checked against the keyword list. Common field names (`sets`, `reps`, `weight`, `group`, `completed`) are identifiers — users may reuse them as variable or parameter names.

Integer vs. Float (Maximal Munch). `80.0` is always one `FLOAT_LIT` token, never split into `INT_LIT(80)`, `.`, `INT_LIT(0)`.

Boolean Literals. `true` and `false` are reserved keywords — they may never be used as identifier names (reliability, Sebesta §6.12).

Comments. Single-line comments (`//`) are discarded entirely by the lexer.

Annotated Token Stream Example

```

1 routine chest_day(sets: int) {
2     exercise bench_press {
3         weight: 80.0
4         completed: false
5     }
6     rest 60 seconds

```

```
7 }

```

Listing 2: Short snippet for token illustration

Lexeme	Type	Lexeme	Type	Lexeme	Type
routine	KEYWORD	exercise	KEYWORD	80.0	FLOAT_LIT
chest_day	IDENT	bench_press	IDENT	completed	IDENT
(	SEP	{	SEP	:	SEP
sets	IDENT	weight	IDENT	false	KEYWORD
:	SEP	:	SEP	rest	KEYWORD
int	KEYWORD	80.0	FLOAT_LIT	60	INT_LIT
)	SEP	completed	IDENT	seconds	KEYWORD

4.3 Syntax

This section presents the complete grammar of PumpScript in EBNF notation (Sebesta §3.1–3.2). The grammar is unambiguous: operator precedence is encoded structurally, and the dangling-else ambiguity is resolved by the optional ["else" ...] construct.

EBNF Notation Guide

Symbol	Meaning
<code><name></code>	Non-terminal (a grammar rule)
<code>"keyword"</code>	Terminal (literal token in source)
IDENT	Any identifier token
INT_LIT	Any integer literal
FLOAT_LIT	Any float literal
STRING_LIT	Any string literal
{ ... }	Zero or more repetitions
[...]	Optional (zero or one occurrence)
	Alternation (or)

Complete Grammar

```
(* Top-level *)
<program>      = { <routine_decl> | <statement> }

(* Routine declaration *)
<routine_decl> = "routine" IDENT "(" [ <param_list> ] ")" <block>
<param_list>  = <param> { "," <param> }
<param>       = IDENT ":" <type>
<type>        = "int" | "float" | "string" | "bool"

(* Block *)
<block>       = "{" { <statement> } "}"

(* Statements *)
<statement>   = <exercise_block>
               | <rest_stmt>
               | <repeat_stmt>
               | <if_stmt>
               | <routine_call>

(* Exercise record: domain-specific construct *)
<exercise_block> = "exercise" IDENT "{" { <field> } "}"
<field>         = IDENT ":" <expr>

(* Domain-specific: rest period between exercises *)
<rest_stmt>     = "rest" INT_LIT "seconds"

(* Control structures *)
<repeat_stmt>   = "repeat" INT_LIT "times" <block>
<if_stmt>       = "if" <expr> <block> [ "else" <block> ]

(* Routine call *)
<routine_call> = IDENT "(" [ <arg_list> ] ")"
<arg_list>     = <expr> { "," <expr> }
```

```

(* Expressions: precedence encoded bottom-up *)
<expr>          = <comparison> { ("==" | "!=") <comparison> }
<comparison>    = <term> { (> | "<" | ">=" | "<=") <term> }
<term>          = <factor> { ("+" | "-") <factor> }
<factor>        = <unary> { ("*" | "/") <unary> }
<unary>         = "-" <primary> | <primary>
<primary>       = INT_LIT | FLOAT_LIT | STRING_LIT
                  | "true" | "false" | IDENT
                  | "(" <expr> ")"

```

Listing 3: PumpScript EBNF grammar

Ambiguity Resolution

Operator Precedence

Level	Operators	Rule (deepest = tightest binding)
Highest	- (unary)	<code><unary></code>
	* /	<code><factor></code>
	+ -	<code><term></code>
	> < >= <=	<code><comparison></code>
Lowest	== !=	<code><expr></code>

$2 + 3 * 4$ is always $2 + (3*4) = 14$, never $(2+3)*4 = 20$.

Associativity

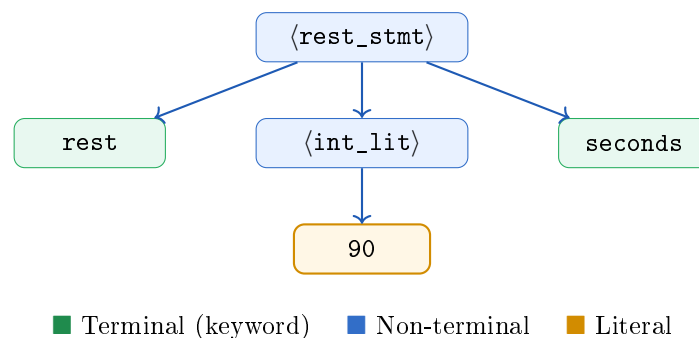
All binary operators are **left-associative** via the iterative `{ ... }` construct. So $10 - 3 - 2 = (10-3)-2 = 5$.

Dangling Else

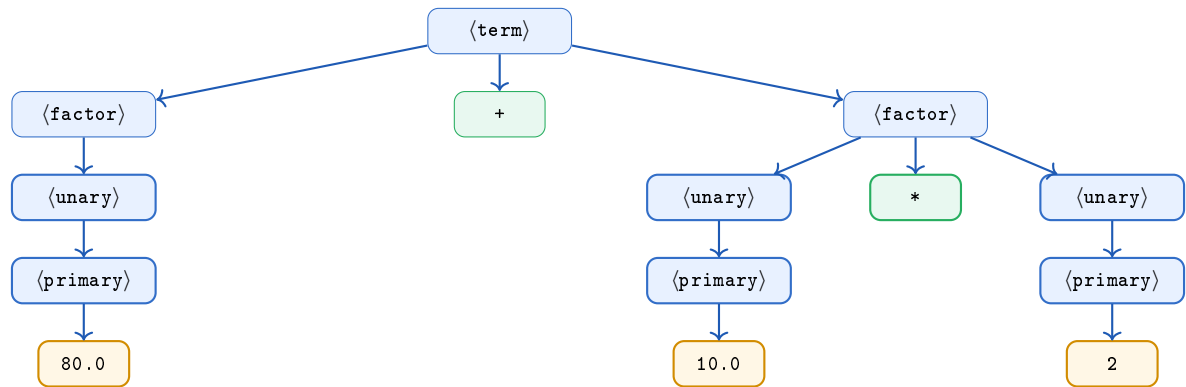
Resolved by `[ "else" ... ]`: `else` always binds to the nearest preceding `if`.

Parse Tree Examples

Example 1 — rest 90 seconds



Example 2 — $80.0 + 10.0 * 2$ (precedence)



$10.0 * 2$ groups under **<factor>** (deeper rule) confirming $*$ binds tighter than $+$.

4.6 Names, Binding, Scope, and Lifetime

4.6.1 Legal Identifiers

Decision: Identifier Rules

An identifier must begin with a letter (a-z, A-Z) or underscore (_), followed by zero or more letters, digits, or underscores.

Regular expression: `[a-zA-Z_][a-zA-Z0-9_]*`

Valid	Invalid	Why Invalid
squat	3sets	Starts with digit
leg_day	bench-press	Hyphen not allowed
chest_fly	routine	Reserved keyword
myRoutine	true	Reserved keyword
_warmup	sets!	Special char not allowed

Reserved keywords: `routine`, `exercise`, `rest`, `repeat`, `times`, `if`, `else`, `seconds`, `true`, `false`, `int`, `float`, `string`, `bool`.

4.6.2 Compile-time vs. Run-time Bindings

What is bound	When	Example
Routine names	Compile-time	<code>routine chest_day</code> known before execution
Parameter types	Compile-time	<code>sets: int</code> verified by type checker
Operator meanings	Compile-time	<code>+</code> on int vs float resolved by type
Parameter values	Run-time	<code>chest_day(4) → sets=4</code> at call
Field values	Run-time	<code>weight: 80.0</code> evaluated when block executes
Conditional branch	Run-time	Which <code>if</code> branch runs depends on runtime value

4.6.3 Scoping Rule: Static (Lexical) Scoping

Decision: Static Scoping

PumpScript uses **static (lexical) scoping** (Sebesta §5.5). The scope of every name is determined by where it appears in the source text, not by the call chain at run time.

Global Scope — routine declarations visible everywhere

```

Local Scope of chest_day(sets: int)
  exercise bench_press {
    sets: sets  // OK -- sets is in scope
    weight: 80.0
  }
routine leg_day(sets: int) { ... }  // visible globally
chest_day(4)  // call from global scope
sets // ERROR: sets not visible here

```

Dynamic Scoping — Why PumpScript Rejects It

Under dynamic scoping, a name resolves to the most recent binding on the call stack:

```

routine a(sets: int) {
  exercise squat { sets: sets }  // Which "sets"?
}
routine b(sets: int) { a(4) }    // b's "sets" would bleed into a!
b(10)  // squat would get sets=10, not sets=4 -- unexpected!

```

A fitness coach would never expect this. Static scoping prevents it entirely.

4.6.4 Variable Lifetime**Decision: Lifetime Rules**

- **Routine declarations** → **static lifetime**: exist for the entire program execution.
- **Parameters and local bindings** → **stack-dynamic lifetime**: created on call, destroyed on return.
- **Heap-dynamic allocation** is *not supported*.

Time
↑

Program End — static bindings released
chest_day, leg_day definitions removed from memory

leg_day(3) called — new stack frame
sets = 3 (fresh binding, independent of chest_day's sets)
exercise squat ... rest 90 ... exercise lunges

chest_day returns — stack frame destroyed
sets = 4 no longer exists in memory

chest_day(4) called — stack frame created
sets = 4 (stack-dynamic, bound at call)
exercise bench_press ... rest 60 ... exercise chest_fly

Program Start — routine declarations bound (static)
chest_day → defined leg_day → defined

Summary

Question (Sebesta Ch. 5)	Decision	Justification
Legal identifiers	<code>[a-zA-Z_][a-zA-Z0-9_]</code>	Familiar to Python/C users
Compile-time bindings	Names, types, operators	Enables static type checking
Run-time bindings	Parameter values, field values	Depend on actual call arguments
Scoping rule	Static (lexical)	Predictable; dynamic causes interference
Local variable lifetime	Stack-dynamic	No heap; automatic per-call
Routine declaration lifetime	Static	Available throughout program
